The database in an expert system attempts to capture the knowledge ("elicit the expertise") of a human expert in a particular field, including both the facts known to the expert and the expert's reasoning path in reaching conclusions from those facts. The completed expert system not only simulates the human expert's actions but can be questioned to reveal why it made certain choices and not others.

Expert systems have been built that simulate a medical specialist's diagnosis from a patient's symptoms, a factory manager's decisions regarding valve control in a chemical plant based on sensor readings, the decisions of a fashion buyer for a retail store based on market research, the choices made by a consultant specifying a computer system configuration based on customer needs, and many more. The challenging part of building an expert system lies in extracting all pertinent facts and rules from the human expert.

SECTION 1.5 REVIEW

TECHNIQUES

- Formulate Prolog-like facts and rules.
- Formulate Prolog-like queries.
- Determine the answer(s) to a query using a Prolog database.

MAIN IDEA

- A declarative language incorporates predicate wffs and rules of inference to draw conclusions from hypotheses. The elements of such a language are based on predicate logic rather than instructions that carry out an algorithm.

EXERCISES 1.5

Exercises 1–8 refer to the database of Example 39; find the results of the query in each case.

1. ?animal(wildcat)
2. ?plant(raccoon)
3. ?*eat*(bear, littlefish)
4. ?*eat*(fox, rabbit)
5. ?eat(raccoon, X)
6. ?*eat*(X, grass)
7. ?*eat*(bear, X) and *eat*(X, rabbit)
8. ?*prey*(X) and not *eat*(fox, X)
9. Formulate a Prolog rule that defines "herbivore" to add to the database of Example 39.
10. If the rule of Exercise 9 is included in the database of Example 39, what is the response to the query

    ?*herbivore*(X)

11. After *infoodchain* is added to the database of Example 39, add the facts *eat*(wolf, fox) and *eat*(wolf, deer). What is the result of the query

    ?*eat*(wolf, X) and not *eat*(X, grass)

12. After the modifications in Exercise 11, what is the result of the query

    ?infoodchain(wolf, X)

13. A Prolog database contains the following, where boss(X, Y) means "X is Y's boss" and supervisor(X, Y) means "X is Y's supervisor":

    boss(mike, joan)
    boss(judith, mike)
    boss(anita, judith)
    boss(judith, kim)
    boss(kim, enrique)
    boss(anita, sam)
    boss(enrique, jefferson)
    boss(mike, hamal)

    supervisor(X, Y) <= *boss*(X, Y)
    supervisor(X, Y) <= *boss*(X, Z) and *supervisor*(Z, Y)

    Find the results of the following queries:

    a. ?*boss*(X, sam)
    b. ?*boss*(judith, X)
    c. ?*supervisor*(anita, X)

14. Using the Prolog database from Exercise 13, what are the results of the following queries?
    a. ?boss(hamal, X)
    b. ?supervisor(X, kim)

15. Suppose a Prolog database exists that gives information about authors and the books they have written. Books are classified as fiction, biography, or reference.
    a. Write a query to ask whether Mark Twain wrote *Hound of the Baskervilles*.
    b. Write a query to find all books written by William Faulkner.
    c. Formulate a rule to define nonfiction authors.
    d. Write a query to find all nonfiction authors.

16. Suppose a Prolog database exists that gives information about states and capital cities. Some cities are big, others small. Some states are eastern, others are western.
    a. Write a query to find all the small capital cities.
    b. Write a query to find all the states with small capital cities.
    c. Write a query to find all the eastern states with big capital cities.
    d. Formulate a rule to define cosmopolitan cities as big capitals of western states.
    e. Write a query to find all the cosmopolitan cities.

17. Suppose a Prolog database exists that gives information about a family. Predicates of *male*, *female*, and *parentof* are included.
    a. Formulate a rule to define *fatherof*.
    b. Formulate a rule to define *daughterof*.
    c. Formulate a recursive rule to define *ancestorof*.

18. Suppose a Prolog database exists that gives information about the parts in an automobile engine. Predicates of *big*, *small*, and *partof* are included.
    a. Write a query to find all small items that are part of other items.
    b. Write a query to find all big items that have small subitems.
    c. Formulate a recursive rule to define *componentof*.

19. Suppose a Prolog database exists that gives information about the ingredients in the menu items of a restaurant. Predicates of *dry*, *liquid*, *perishable*, and *ingredientof* are included.
    a. Write a query to find all the dry ingredients of other ingredients.
    b. Write a query to find all perishable ingredients that contain liquid subingredients.
    c. Formulate a recursive rule to define *foundin*.
20. Suppose a Prolog database exists that gives information about flights for AA (Always Airborne) airline. Predicates of *city* and *flight* are included. Here *flight*(*X*, *Y*) means that AA has a direct (nonstop) flight from city *X* to city *Y*.
    a. Write a query to find all cities you can get to on a direct flight from Indianapolis.
    b. Write a query to find all cities that have direct flights to San Francisco.
    c. Formulate a recursive rule to define *route* where *route*(*X*, *Y*) means that you can get from city *X* to city *Y* using AA but it might not be a direct flight.

Exercises 21–22 refer to a "Toy Prolog interpreter" that can be found online at http://www.csse.monash.edu.au/~lloyd/tildeLogic/Prolog.toy. The syntax used in this section matches that of this online version except that in the online version each statement and query must end with a period. Here is a shortened version of Example 39 as entered into the code window, followed by the response to the query:

```
eat(bear, fish).
eat(fish, littlefish).
eat(littlefish, algae).
eat(raccoon, fish).
eat(bear, raccoon).
eat(bear, fox).
animal(bear).
animal(fish).
animal(littlefish).
animal(raccoon).
animal(fox).
prey(X) <= eat(Y, X) and animal(X).
?prey(X).

--- running ---

prey(fish) yes
prey(littlefish) yes
prey(fish) yes
prey(raccoon) yes
prey(fox) yes
```

In addition, you can look at and run the online sample program to be sure you understand the syntax rules.

21. Using the online Toy Prolog program, enter the Prolog database of Exercise 13. Run the queries from Exercises 13 and 14 and compare the results with your previous answers.
22. Using the online Toy Prolog program, create a database for Exercise 20. Run the queries from Exercise 20. Also run a query using the *route* predicate.

SECTION 1.6 PROOF OF CORRECTNESS

As our society becomes ever more dependent on computers, it is more and more important that the programs computers run are reliable and error-free. **Program verification** attempts to ensure that a computer program is correct.

"Correctness" has a narrower definition here than in everyday usage. A program is **correct** if it behaves in accordance with its specifications. However, this does not necessarily mean that the program solves the problem that it was intended to solve; the program's specifications may be at odds with or not address all aspects of a client's requirements. **Program validation**, which we won't discuss further, attempts to ensure that the program indeed meets the client's original requirements. In a large program development project "program V & V" or "software quality assurance" is considered so important that a group of people separate from the programmers is often designated to carry out the associated tasks.

Program verification may be approached both through program testing and through *proof of correctness*. **Program testing** seeks to show that particular input values produce acceptable output values. Program testing is a major part of any software development effort, but it is well-known folklore that "testing can prove the presence of errors but never their absence." If a test run under a certain set of conditions with a certain set of input data reveals a "bug" in the code, then the bug can be corrected. But except for rather simple programs, multiple tests that reveal no bugs do not guarantee that the code is bug-free, that there is not some error lurking in the code waiting to strike under the right circumstances.

As a complement to testing, computer scientists have developed a more mathematical approach to "prove" that a program is correct. **Proof of correctness** uses the techniques of a formal logic system to prove that if the input variables satisfy certain specified predicates or properties, the output variables produced by executing the program satisfy other specified properties.

To distinguish between proof of correctness and program testing, consider a program to compute the length c of the hypotenuse of a right triangle, given positive values a and b for the lengths of the legs. Proving the program correct would establish that whenever a and b satisfy the predicates $a > 0$ and $b > 0$, then after the program is executed, the predicate $a^2 + b^2 = c^2$ is satisfied. Testing such a program would require taking various specific values for a and b, computing the resulting c, and checking that $a^2 + b^2$ equals c^2 in each case. However, only representative values for a and b can be tested, not all possible values.

Again, testing and proof of correctness are complementary aspects of program verification. All programs undergo program testing; they may or may not undergo proof of correctness as well. Proof of correctness is labor-intensive, hence expensive; it generally is applied only to small and critical sections of code rather than to the entire program.

Assertions

Describing proof of correctness more formally, let us denote by X an arbitrary collection of input values to some program or program segment P. The actions of P transform X into a corresponding group of output values Y; the notation $Y = P(X)$ suggests that the Y values depend on the X values through the actions of program P.

A predicate $Q(X)$ describes conditions that the input values are supposed to satisfy. For example, if a program is supposed to find the square root of a positive number, then X consists of one input value, x, and $Q(x)$ might be "$x > 0$."

A predicate R describes conditions that the output values are supposed to satisfy. These conditions will often involve the input values as well, so R has the form $R(X, Y)$ or $R[X, P(X)]$. In our square root case, if y is the single output value, then y is supposed to be the square root of x, so $R(x, y)$ would be "$y^2 = x$." Program P is correct if the implication

$$(\forall X)(Q(X) \rightarrow R[X, P(X)]) \tag{1}$$

is valid. In other words, whenever Q is true about the input values, R should be true about the input and output values. For the square root case, (1) is

$$(\forall x)(x > 0 \rightarrow [P(x)]^2 = x)$$

The implication (1) is standard predicate wff notation, but the traditional program correctness notation for (1) is

$$\{Q\}P\{R\} \tag{2}$$

$\{Q\}P\{R\}$ is called a **Hoare triple**, named for the British computer scientist Anthony Hoare. Condition Q is called the **precondition** for program P, and condition R is the **postcondition**. In the Hoare notation, the universal quantifier does not explicitly appear; it is understood.

Rather than simply having an initial predicate and a final predicate, a program or program segment is broken down into individual statements s_i, with predicates inserted between statements as well as at the beginning and end. These predicates are also called **assertions** because they assert what is supposed to be true about the program variables at that point in the program. Thus we have

$$\begin{array}{c} \{Q\} \\ s_0 \\ \{R_1\} \\ s_1 \\ \{R_2\} \\ \vdots \\ s_{n-1} \\ \{R\} \end{array}$$

where $Q, R_1, R_2, \ldots, R_n = R$ are assertions. The intermediate assertions are often obtained by working backward from the output assertion R.

P is provably correct if each of the following implications holds:

$$\begin{array}{l} \{Q\}s_0\{R_1\} \\ \{R_1\}s_1\{R_2\} \\ \{R_2\}s_2\{R_3\} \\ \quad\vdots \\ \{R_{n-1}\}s_{n-1}\{R\} \end{array}$$

A proof of correctness for P consists of producing this sequence of valid implications, that is, producing a proof sequence of predicate wffs. Some new rules of inference can be used, based on the nature of the program statement s_i.

Assignment Rule

Suppose that statement s_i is an assignment statement of the form $x = e$, that is, the variable x takes on the value of e, where e is some expression. The Hoare triple to prove correctness of this one statement has the form

$$\{R_i\}\ x = e\ \{R_{i+1}\}$$

For this triple to be valid, the assertions R_i and R_{i+1} must be related in a particular way.

EXAMPLE 41 Consider the following assignment statement together with the given precondition and postcondition:

$$\{x - 1 > 0\}$$
$$x = x - 1$$
$$\{x > 0\}$$

For every x, if $x - 1 > 0$ before the statement is executed (note that this says that $x > 1$), then after the value of x is reduced by 1, it will be the case that $x > 0$. Therefore,

$$\{x - 1 > 0\}\ x = x - 1\ \{x > 0\}$$

is valid.

In Example 41, we just reasoned our way through the validity of the wff represented by the Hoare triple. The point of predicate logic is to allow us to determine validity in a more mechanical fashion by the application of rules of inference. (After all, we don't want to just "reason our way through" the entire program to convince ourselves of its correctness; the programmer already did that when the program was written!)

The appropriate rule of inference for assignment statements is the **assignment rule**, given in Table 1.18. It says that if the precondition and postcondition are appropriately related, the Hoare triple can be inserted at any time in a proof sequence without having to be inferred from something earlier in the proof sequence. This makes the Hoare triple for an assignment statement akin to a hypothesis in our previous proofs. And what is the relationship? In the postcondition, locate all instances of the variable to which an assignment is being made in the assignment statement right above the postcondition. For each of those instances, substitute the expression being assigned. The result will be the precondition.

TABLE 1.18

From	Can Derive	Name of Rule	Restrictions on Use
	$\{R_i\}s_i\{R_{i+1}\}$	assignment	1. s_i has the form $x = e$. 2. R_i is R_{i+1} with e substituted everywhere for x.

EXAMPLE 42 For the case of Example 41,

$$\{x - 1 > 0\}$$
$$x = x - 1$$
$$\{x > 0\}$$

the triple

$$\{x - 1 > 0\}\ x = x - 1\ \{x > 0\}$$

is valid by the assignment rule. The postcondition is

$$x > 0$$

Substituting $x - 1$ for x throughout the postcondition results in

$$x - 1 > 0 \quad \text{or} \quad x > 1$$

which is the precondition. Here we didn't have to think at all; we just checked that the assignment rule had been followed.

Certainly Example 41 seems easier than Example 42, and for such a trivial case you may be tempted to skip use of the assignment inference rule and just talk your way through the code. Resist this temptation. For one thing, real-world usage isn't this trivial. But more to the point, just as in our previous formal logic systems, you want to rely on the rules of inference instead of on some possibly flawed thought process.

PRACTICE 31 According to the assignment rule, what should be the precondition in the following program segment?

{precondition}
$x = x - 2$
$\{x = y\}$

REMINDER
To use the assignment rule, work from the bottom to the top.

Because the assignment rule tells us what a precondition should look like based on what a postcondition looks like, a proof of correctness often begins with the final desired postcondition and works its way back up through what the earlier assertions should look like according to the assignment rule. Once it has been determined what the topmost assertion must be, a check is done to see that this assertion is really true.

EXAMPLE 43 Verify the correctness of the following program segment to exchange the values of x and y:

temp $= x$
$x = y$
$y =$ temp

At the beginning of this program segment, x and y have certain values. Thus we may express the actual precondition as $x = a$ and $y = b$. The desired postcondition is then $x = b$ and $y = a$. Using the assignment rule, we can work backward from the postcondition to find the earlier assertions (read the following from the bottom to the top).

$\{y = b, x = a\}$
  temp $= x$
$\{y = b,$ temp $= a\}$
  $x = y$
$\{x = b,$ temp $= a\}$
  $y =$ temp
$\{x = b, y = a\}$

The first assertion agrees with the precondition; the assignment rule, applied repeatedly, assures us that the program segment is correct.

PRACTICE 32 Verify the correctness of the following program segment with the precondition and postcondition shown:

$\{x = 3\}$
  $y = 4$
  $z = x + y$
$\{z = 7\}$

Sometimes the necessary precondition is trivially true, as shown in the next example.

EXAMPLE 44 Verify the correctness of the following program segment to compute $y = x - 4$.

$y = x$
$y = y - 4$

Here the desired postcondition is $y = x - 4$. Using the assignment rule to work backward from the postcondition, we get (again, read bottom to top)

$\{x - 4 = x - 4\}$
  $y = x$
$\{y - 4 = x - 4\}$
  $y = y - 4$
$\{y = x - 4\}$

The precondition is always true; therefore, by the assignment rule, each successive assertion, including the postcondition, is true.

Conditional Rule

A **conditional statement** is a program statement of the form

if condition B **then**
  P_1
else
  P_2
end if

When this statement is executed, a condition B that is either true or false is evaluated. If B is true, program segment P_1 is executed, but if B is false, program segment P_2 is executed.

A **conditional rule of inference**, shown in Table 1.19, determines when a Hoare triple

$$\{Q\}s_i\{R\}$$

can be inserted in a proof sequence if s_i is a conditional statement. The Hoare triple is inferred from two other Hoare triples. One of these says that if Q is true and B is true and program segment P_1 is executed, then R holds; the other says that if Q is true and B is false and program segment P_2 is executed, then R holds. This simply says that each branch of the conditional statement must be proved correct.

TABLE 1.19

From	Can Derive	Name of Rule	Restrictions on Use

$\{Q \wedge B\}\ P_1\ \{R\}$, $\{Q \wedge B'\}\ P_2\ \{R\}$	$\{Q\}s_i\{R\}$	conditional	s_i has the form
			if condition B **then**
			  P_1
			else
			  P_2
			end if

EXAMPLE 45 Verify the correctness of the following program segment with the precondition and postcondition shown.

$\{n = 5\}$
  if $n >= 10$ **then**
    $y = 100$
  else
    $y = n + 1$
  end if
$\{y = 6\}$

Here the precondition is $n = 5$, and the condition B to be evaluated is $n >= 10$. In order to apply the conditional rule, we must first prove that

$$\{Q \wedge B\}\ P_1\ \{R\}$$

or

$$\{n = 5 \text{ and } n \geq 10\}\ \text{y} = 100\ \{y = 6\}$$

holds. Remember that this stands for an implication, which will be true because its antecedent, $n = 5$ and $n \geq 10$, is false. We must also show that

$$\{Q \wedge B'\}\ P_2\ \{R\}$$

or

$$\{n = 5 \text{ and } n < 10\}\ y = n + 1\ \{y = 6\}$$

holds. Working back from the postcondition, using the assignment rule, we get

$$\begin{array}{c}\{n + 1 = 6 \text{ or } n = 5\}\\ y = n + 1\\ \{y = 6\}\end{array}$$

Thus

$$\{n = 5\}\ y = n + 1\ \{y = 6\}$$

is true by the assignment rule and therefore

$$\{n = 5 \text{ and } n < 10\}\ y = n + 1\ \{y = 6\}$$

is also true because the condition $n < 10$ adds nothing new to the assertion. The conditional rule allows us to conclude that the program segment is correct.

PRACTICE 33 Verify the correctness of the following program segment with the precondition and postcondition shown.

```
{x = 4}
   if x < 5 then
      y = x − 1
   else
      y = 7
   end if
{y = 3}
```

EXAMPLE 46 Verify the correctness of the following program segment to compute max(x, y), the maximum of two distinct values x and y.

```
{x ≠ y}
   if x >= y then
      max = x
   else
      max = y
   end if
```

The desired postcondition reflects the definition of the maximum, ($x > y$ and max $= x$) or ($x < y$ and max $= y$). The two implications to prove are

$$\{x \neq y \text{ and } x \geq y\} \text{ max} = x \ \{(x > y \text{ and max} = x) \text{ or } (x < y \text{ and max} = y)\}$$

and

$$\{x \neq y \text{ and } x < y\} \text{ max} = y \ \{(x > y \text{ and max} = x) \text{ or } (x < y \text{ and max} = y)\}$$

Using the assignment rule on the first case (substituting x for max in the postcondition) would give the precondition

$$(x > y \wedge x = x) \vee (x < y \wedge x = y)$$

Since the second disjunct is always false, this is equivalent to

$$(x > y \wedge x = x)$$

which in turn is equivalent to

$$x > y \qquad \text{or} \qquad x \neq y \text{ and } x \geq y$$

The second implication is proved similarly. ●

In Chapter 2, we will see how to verify correctness for a loop statement, where a section of code can be repeated many times.

As we have seen, proof of correctness involves a lot of detailed work. It is a difficult tool to apply to large programs that already exist. It is generally easier to prove correctness while the program is being developed. Indeed, the list of assertions from beginning to end specifies the intended behavior of the program and can be used early in its design. In addition, the assertions serve as valuable documentation after the program is complete.

SECTION 1.6 REVIEW

TECHNIQUES

- Verify the correctness of a program segment that includes assignment statements.
- Verify the correctness of a program segment that includes conditional statements.

MAIN IDEA

- A formal system of rules of inference can be used to prove the correctness of program segments.

EXERCISES 1.6

In the following exercises, * denotes multiplication.

1. According to the assignment rule, what is the precondition in the following program segment?

   {precondition}
   $x = x + 1$
   $\{x = y - 1\}$

2. According to the assignment rule, what is the precondition in the following program segment?
   {precondition}
   $x = 2 * x$
   $\{x > y\}$
3. According to the assignment rule, what is the precondition in the following program segment?
   {precondition}
   $x = 3 * x - 1$
   $\{x = 2 * y - 1\}$
4. According to the assignment rule, what is the precondition in the following program segment?
   {precondition}
   $y = 3x + 7$
   $\{y = x + 1\}$
5. Verify the correctness of the following program segment with the precondition and postcondition shown.
   $\{x = 1\}$
   $y = x + 3$
   $y = 2 * y$
   $\{y = 8\}$
6. Verify the correctness of the following program segment with the precondition and postcondition shown.
   $\{x > 0\}$
   $y = x + 2$
   $z = y + 1$
   $\{z > 3\}$
7. Verify the correctness of the following program segment with the precondition and postcondition shown.
   $\{x = 0\}$
   $z = 2 * x + 1$
   $y = z - 1$
   $\{y = 0\}$
8. Verify the correctness of the following program segment with the precondition and postcondition shown.
   $\{x < 8\}$
   $z = x - 1$
   $y = z - 5$
   $\{y < 2\}$
9. Verify the correctness of the following program segment to compute $y = x(x - 1)$.
   $y = x - 1$
   $y = x * y$
10. Verify the correctness of the following program segment to compute $y = 2x + 1$.
    $y = x$
    $y = y + y$
    $y = y + 1$
11. Verify the correctness of the following program segment with the precondition and postcondition shown.
    $\{y = 0\}$
    if $y < 5$ **then**
      $y = y + 1$
    else
      $y = 5$
    end if
    $\{y = 1\}$

12. Verify the correctness of the following program segment with the precondition and postcondition shown.

$\{x = 7\}$
if $x <= 0$ **then**
$y = x$
else
$y = 2 * x$
end if
$\{y = 14\}$

13. Verify the correctness of the following program segment with the precondition and postcondition shown.

$\{x \neq 0\}$
if $x > 0$ **then**
$y = 2 * x$
else
$y = (-2) * x$
end if
$\{y > 0\}$

14. Verify the correctness of the following program segment to compute min(x, y), the minimum of two distinct values x and y.

$\{x \neq y\}$
if $x <= y$ **then**
min $= x$
else
min $= y$
end if

15. Verify the correctness of the following program segment to compute $|x|$, the absolute value of x, for a nonzero number x.

$\{x \neq 0\}$
if $x >= 0$ **then**
abs $= x$
else
abs $= -x$
end if

16. Verify the correctness of the following program segment with the assertions shown.

$\{z = 3\}$
$x = z + 1$
$y = x + 2$
$\{y = 6\}$
if $y > 0$ **then**
$z = y + 1$
else
$z = 2 * y$
end if
$\{z = 7\}$

CHAPTER 1 REVIEW

TERMINOLOGY

algorithm (p. 12)
antecedent (p. 3)
assertion (p. 86)
assignment rule (p. 87)
binary connective (p. 3)
binary predicate (p. 40)
complete formal logic system (p. 27)
conclusion (p. 25)
conditional rule of inference (p. 90)
conditional statement (p. 90)
conjunct (p. 2)
conjunction (p. 2)
consequent (p. 3)
contradiction (p. 8)
correct formal logic system (p. 27)
correct program (p. 85)
De Morgan's laws (p. 10)
declarative language (p. 73)
depth-first search (p. 81)
derivation rule (p. 27)
descriptive language (p. 73)
disjunct (p. 3)
disjunction (p. 3)
domain (p. 41)
dual of an equivalence (p. 9)
equivalence (p. 3)
equivalence rule (p. 28)
equivalent wffs (p. 8)
existential generalization (p. 62)
existential instantiation (p. 60)
existential quantifier (p. 40)
expert system (p. 81)
free variable (p. 42)
Hoare triple (p. 86)
Horn clause (p. 76)
hypothesis (p. 25)
implication (p. 3)
inference rule (p. 29)
interpretation (p. 41)
knowledge-based system (p. 81)
logical connective (p. 2)
main connective (p. 6)
n-ary predicate (p. 40)
negation (p. 4)
postcondition (p. 86)
precondition (p. 86)
predicate (p. 39)
predicate logic (p. 58)
predicate wff (p. 41)
procedural language (p. 73)
program testing (p. 85)
program validation (p. 85)
program verification (p. 84)
Prolog database (p. 73)
Prolog fact (p. 73)
Prolog rule (p. 75)
proof of correctness (p. 85)
proof sequence (p. 27)
proposition (p. 2)
propositional calculus (p. 25)
propositional logic (p. 25)
propositional wff (p. 25)
pseudocode (p. 12)
recursive definition (p. 79)
resolution (p. 76)
rule-based system (p. 81)
scope (p. 41)
statement (p. 2)
statement letter (p. 2)
statement logic (p. 25)
tautology (p. 8)
ternary predicate (p. 40)
unary connective (p. 3)
unary predicate (p. 40)
universal generalization (p. 61)
universal instantiation (p. 59)
universal quantifier (p. 39)
valid argument (p. 26, 58)
valid predicate wff (p. 48)
well-formed formula (wff) (p. 6)

SELF TEST

Answer the following true–false questions without looking back in the chapter.

Section 1.1

1. A contradiction is any propositional wff that is not a tautology.
2. The disjunction of any propositional wff with a tautology has the truth value true.
3. Algorithm *TautologyTest* determines whether any propositional wff is a tautology.
4. Equivalent propositional wffs have the same truth values for every truth value assignment to the components.
5. One of De Morgan's laws states that the negation of a disjunction is the disjunction of the negations (of the disjuncts).

Section 1.2

1. An equivalence rule allows one wff to be substituted for another in a proof sequence.
2. If a propositional wff can be derived using modus ponens, then its negation can be derived using modus tollens.
3. Propositional logic is complete because every tautology is provable.
4. A valid argument is one in which the conclusion is always true.
5. The deduction method applies when the conclusion is an implication.

Section 1.3

1. A predicate wff that begins with a universal quantifier is universally true, that is, true in all interpretations.
2. In the predicate wff $(\forall x)P(x, y)$, y is a free variable.
3. An existential quantifier is usually found with the conjunction connective.
4. The domain of an interpretation consists of the values for which the predicate wff defined on that interpretation is true.
5. A valid predicate wff has no interpretation in which it is false.

Section 1.4

1. The inference rules of predicate logic allow existential and universal quantifiers to be added or removed during a proof sequence.
2. Existential instantiation should be used only after universal instantiation.
3. $P(x) \wedge (\exists x)Q(x)$ can be deduced from $(\forall x)[P(x) \wedge (\exists y)Q(y)]$ using universal instantiation.
4. Every provable wff of propositional logic is also provable in predicate logic.
5. A predicate wff that is not valid cannot be proved using predicate logic.

Section 1.5

1. A Prolog rule describes a predicate.
2. Horn clauses are wffs consisting of single negated predicates.
3. Modus ponens is a special case of Prolog resolution.
4. A Prolog recursive rule is a rule of inference that is used more than once.
5. A Prolog inference engine applies its rule of inference without guidance from either the programmer or the user.

Section 1.6

1. A provably correct program always gives the right answers to a given problem.
2. If an assertion after an assignment statement is $y > 4$, then the precondition must be $y \geq 4$.
3. Proof of correctness involves careful development of test data sets.
4. Using the conditional rule of inference in proof of correctness involves proving that two different Hoare triples are valid.
5. The assertions used in proof of correctness can also be used as a program design aid before the program is written, and as program documentation.

ON THE COMPUTER

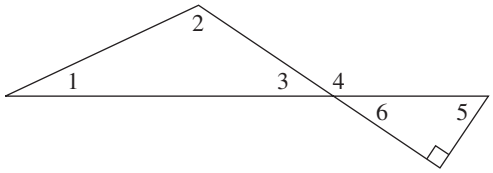
For Exercises 1–5, write a computer program that produces the desired output from the given input.

1. *Input*: Truth values for two statement letters A and B
   Output: Corresponding truth values (appropriately labeled, of course) for
   $$A \wedge B, A \vee B, A \rightarrow B, A \leftrightarrow B, A'$$
2. *Input*: Truth values for two statement letters A and B
   Output: Corresponding truth values for the wffs
   $$A \rightarrow B' \text{ and } B' \wedge [A \vee (A \wedge B)]$$
3. *Input*: Truth values for three statement letters A, B, and C
   Output: Corresponding truth values for the wffs
   $$A \vee (B \wedge C') \rightarrow B' \text{ and } A \vee C' \leftrightarrow (A \vee C)'$$
4. *Input:* Truth values for three statement letters A, B, and C, and a representation of a simple propositional wff. Special symbols can be used for the logical connectives, and postfix notation can be used; for example,
   $$A\ B \wedge C \vee \text{ for } (A \wedge B) \vee C$$
   or
   $$A'\ B \wedge \text{ for } A' \wedge B$$
   Output: Corresponding truth value of the wff
5. *Input*: Representation of a simple propositional wff as in the previous exercise
   Output: Decision on whether the wff is a tautology
6. Using the online Toy Prolog program that can be found at http://www.csse.monash.edu.au/~lloyd/tildeLogic/Prolog.toy, enter the Prolog database of Example 39 and perform the queries there. Note that each database entry requires a period. Also add the recursive rule for *infoodchain* and perform the query
   ?*infoodchain*(bear, *Y*)

62. If n, m, and p are integers and $n \mid mp$, then $n \mid m$ or $n \mid p$.
63. For every prime number n, $n + 4$ is prime.
64. For every positive integer n, $n^2 + n + 3$ is not prime.
65. For n a positive integer, $n > 2$, $n^2 - 1$ is not prime.
66. For every positive integer n, $2^n + 1$ is prime.
67. For every positive integer n, $n^2 + n + 1$ is prime.
68. For n an even integer, $n > 2$, $2^n - 1$ is not prime.
69. The sum of two rational numbers is rational.
70. The product of two rational numbers is rational.
71. The product of two irrational numbers is irrational.
72. The sum of a rational number and an irrational number is irrational.

For Exercises 73–75, use the following facts from geometry and the accompanying figure.

- The interior angles of a triangle sum to 180°.
- Vertical angles (opposite angles formed when two lines intersect) are the same size.
- A straight angle is 180°.
- A right angle is 90°.



73. Prove that the measure of angle 5 plus the measure of angle 3 is 90°.
74. Prove that the measure of angle 4 is the sum of the measures of angles 1 and 2.
75. If angle 1 and angle 5 are the same size, then angle 2 is a right angle.
76. Prove that the sum of the integers from 1 through 100 is 5050. (*Hint:* Instead of actually adding all the numbers, try to make the same clever observation that the German mathematician Karl Friederick Gauss [1777–1855] made as a schoolchild: Group the numbers into pairs, using 1 and 100, 2 and 99, etc.)

SECTION 2.2 INDUCTION

First Principle of Induction

There is one final proof technique especially useful in computer science. To illustrate how the technique works, imagine that you are climbing an infinitely high ladder. How do you know whether you will be able to reach an arbitrarily high rung? Suppose we make the following two assertions about your climbing abilities:

1. You can reach the first rung.
2. Once you get to a rung, you can always climb to the next one up. (Notice that this assertion is an implication.)

If both statement 1 and the implication of statement 2 are true, then by statement 1 you can get to the first rung and therefore by statement 2 you can get to the second; by statement 2 again, you can get to the third rung; by statement 2 again you can get to the fourth; and so on. You can climb as high as you wish. Both assertions here are necessary. If only statement 1 is true, you have no guarantee of getting beyond the first rung, and if only statement 2 is true, you may never be able to get started. Let's assume that the rungs of the ladder are numbered by positive integers—1, 2, 3, and so on.

Now think of a specific property a number might have. Instead of "reaching an arbitrarily high rung," we can talk about an arbitrary, positive integer having that property. We use the shorthand notation $P(n)$ to mean that the positive integer n has the property P. How can we use the ladder-climbing technique to prove that for all positive integers n, we have $P(n)$? The two assertions we need to prove are

1. $P(1)$	(1 has property P.)
2. For any positive integer k, $P(k) \rightarrow P(k+1)$.	(If any number has property P, so does the next number.)

If we can prove both assertions 1 and 2, then $P(n)$ holds for any positive integer n, just as you could climb to an arbitrary rung on the ladder.

The foundation for arguments of this type is the first principle of mathematical induction.

PRINCIPLE **FIRST PRINCIPLE OF MATHEMATICAL INDUCTION**

1. $P(1)$ is true
2. $(\forall k)[P(k) \text{ true} \rightarrow P(k+1) \text{ true}]$

$\rightarrow P(n)$ true for all positive integers n

REMINDER

To prove something true for all $n \geq$ some value, think induction.

The first principle of mathematical induction is an implication. The conclusion is a statement of the form, "$P(n)$ is true for all positive integers n." Therefore, whenever we want to prove that something is true for every positive integer n, it is a good bet that mathematical induction is an appropriate proof technique to use.

To know that the conclusion of this implication is true, we show that the two hypotheses, statements 1 and 2, are true. To prove statement 1, we need only show that property P holds for the number 1, usually a trivial task. Statement 2 is also an implication that must hold for all k. To prove this implication, we assume for an arbitrary positive integer k that $P(k)$ is true and show, based on this assumption, that $P(k+1)$ is true. Therefore $P(k) \rightarrow P(k+1)$ and, using universal generalization, $(\forall k)[P(k) \rightarrow P(k+1)]$. You should convince yourself that assuming that property P holds for the number k is not the same as assuming what we ultimately want to prove (a frequent source of confusion when one first encounters proofs of this kind). It is merely the way to proceed with a direct proof that the implication $P(k) \rightarrow P(k+1)$ is true.

In doing a proof by induction, establishing the truth of statement 1, $P(1)$, is called the **basis**, or **basis step**, for the inductive proof. Establishing the truth of $P(k) \rightarrow P(k+1)$ is called the **inductive step**. When we assume $P(k)$ to be true

so as to prove the inductive step, $P(k)$ is called the **inductive assumption**, or **inductive hypothesis**.

All the proof methods we have talked about in this chapter are techniques for deductive reasoning—ways to prove a conjecture that perhaps was formulated by inductive reasoning. Mathematical induction is also a *deductive* technique, not a method for inductive reasoning (don't get confused by the terminology here). For the other proof techniques, we can begin with a hypothesis and string facts together until we more or less stumble on a conclusion. In fact, even if our conjecture is slightly incorrect, we might see what the correct conclusion is in the course of doing the proof. In mathematical induction, however, we must know right at the outset the exact form of the property $P(n)$ that we are trying to establish. Mathematical induction, therefore, is not an exploratory proof technique—it can only confirm a correct conjecture.

Proofs by Mathematical Induction

Suppose that the ancestral progenitor Smith married and had two children. Let's call these two children generation 1. Now suppose each of those two children had two children; then in generation 2, there were four offspring. This trend continued from generation unto generation. The Smith family tree therefore looks like Figure 2.2. (This figure looks exactly like Figure 1.1b, where we looked at the possible T–F values for n statement letters.)

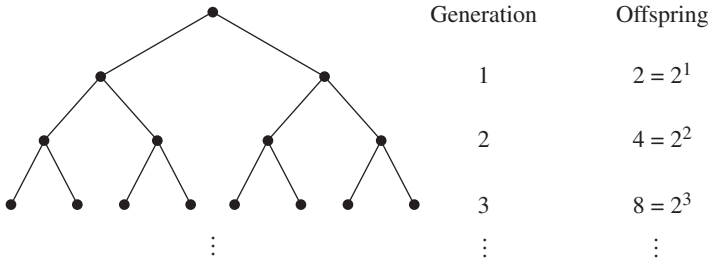


Figure 2.2

It appears that generation n contains 2^n offspring. More formally, if we let $P(n)$ denote the number of offspring at generation n, then we guess that

$$P(n) = 2^n$$

We can use induction to *prove* that our guess for $P(n)$ is correct.

The basis step is to establish $P(1)$, which is the equation

$$P(1) = 2^1 = 2$$

This is true because we are told that Smith had two children. We now assume that our guess is correct for an arbitrary generation k, $k \geq 1$; that is, we assume

$$P(k) = 2^k$$

and try to show that

$$P(k + 1) = 2^{k+1}$$

In this family, each offspring has two children; thus the number of offspring at generation $k + 1$ will be twice the number at generation k, or $P(k + 1) = 2P(k)$. By the inductive assumption, $P(k) = 2^k$, so

$$P(k + 1) = 2P(k) = 2(2^k) = 2^{k+1}$$

and so indeed

$$P(k + 1) = 2^{k+1}$$

This completes our proof. Now that we have set our mind at ease about the Smith clan, we can apply the inductive proof technique to less obvious problems.

EXAMPLE 14 Prove that the equation

$$1 + 3 + 5 + \cdots + (2n - 1) = n^2 \qquad (1)$$

is true for any positive integer n. Here the property $P(n)$ is equation (1). (Notice that $P(n)$ is a property of n, or—in language from Chapter 1—a unary predicate. It is a statement about n, expressed here as an equation. Thus it is incorrect to write something like $P(n) = 1 + 3 + 5 + \cdots + (2n - 1)$.)

The left side of this equation is the sum of all the odd integers from 1 to $2n - 1$. The right side is a formula for the value of this sum. Although we can verify the truth of this equation for any particular value of n by substituting that value for n, we cannot substitute all possible positive integer values. Thus a proof by exhaustion does not work. A proof by mathematical induction is appropriate.

The basis step is to establish $P(1)$, which is equation (1) when n has the value 1. When we substitute 1 for n in the left side of equation (1), we get the sum of all the odd integers starting at 1 and ending at $2(1) - 1 = 1$. The sum of all the odd numbers from 1 to 1 equals 1. When we substitute 1 for n in the formula on the right side of this equation, we get $(1)^2$. Therefore

$$P(1): \quad 1 = 1^2$$

This is certainly true. For the inductive hypothesis, we assume $P(k)$ for an arbitrary positive integer k, which is equation (1) when n has the value k, or

$$P(k): \quad 1 + 3 + 5 + \cdots + (2k - 1) = k^2 \qquad (2)$$

(Note that $P(k)$ is *not* the equation $(2k - 1) = k^2$, which is true only for $k = 1$.) Using the inductive hypothesis, we want to show $P(k + 1)$, which is equation (1) when n has the value $k + 1$, or

$$P(k + 1): \quad 1 + 3 + 5 + \cdots + [2(k + 1) - 1] \stackrel{?}{=} (k + 1)^2 \qquad (3)$$

(The question mark over the equals sign is to remind us that this is the fact we want to prove, as opposed to something we already know.)

The key to an inductive proof is to find a way to relate what we want to show—$P(k+1)$, equation (3)—to what we have assumed—$P(k)$, equation (2). The left side of $P(k+1)$ can be rewritten to show the next-to-last term:

$$1 + 3 + 5 + \cdots + (2k-1) + [2(k+1) - 1]$$

This expression contains the left side of equation (2) as a subexpression. Because we have assumed $P(k)$ to be true, we can substitute the right side of equation (2) for this subexpression. Thus,

$$\begin{aligned} &1 + 3 + 5 + \cdots + [2(k+1) - 1] \\ &\quad = 1 + 3 + 5 + \cdots + (2k-1) + [2(k+1) - 1] \\ &\quad = k^2 + [2(k+1) - 1] \\ &\quad = k^2 + [2k + 2 - 1] \\ &\quad = k^2 + 2k + 1 \\ &\quad = (k+1)^2 \end{aligned}$$

Therefore,

$$1 + 3 + 5 + \cdots + [2(k+1) - 1] = (k+1)^2$$

which verifies $P(k+1)$ and proves that equation (1) is true for any positive integer n.

Table 2.3 summarizes the three steps necessary for a proof using the first principle of induction.

TABLE 2.3

To Prove by First Principle of Induction	
Step 1	Prove base case.
Step 2	Assume $P(k)$.
Step 3	Prove $P(k+1)$.

Any "summation" induction problem works exactly the same way. Write the summation including the next-to-last term, and you will find the left side of the $P(k)$ equation and can use the inductive hypothesis. After that, it's just a question of algebra.

EXAMPLE 15 Prove that

$$1 + 2 + 2^2 + \cdots + 2^n = 2^{n+1} - 1$$

for any $n \geq 1$.

Again, induction is appropriate. $P(1)$ is the equation

$$1 + 2 = 2^{1+1} - 1 \qquad \text{or} \qquad 3 = 2^2 - 1$$

REMINDER

To prove $P(k) \longrightarrow P(k+1)$, you have to discover the $P(k)$ case within the $P(k+1)$ case.

which is true. We take $P(k)$

$$1 + 2 + 2^2 + \cdots + 2^k = 2^{k+1} - 1$$

as the inductive hypothesis and try to establish $P(k+1)$:

$$1 + 2 + 2^2 + \cdots + 2^{k+1} \stackrel{?}{=} 2^{k+1+1} - 1$$

Rewriting the sum on the left side of $P(k+1)$ reveals how the inductive assumption can be used:

$$\begin{aligned} &1 + 2 + 2^2 + \cdots + 2^{k+1} \\ &\quad = 1 + 2 + 2^2 + \cdots + 2^k + 2^{k+1} \\ &\quad = 2^{k+1} - 1 + 2^{k+1} && \text{(from the inductive hypothesis P}(k)\text{)} \\ &\quad = 2(2^{k+1}) - 1 && \text{(adding like terms)} \\ &\quad = 2^{k+1+1} - 1 \end{aligned}$$

Therefore,

$$1 + 2 + 2^2 + \cdots + 2^{k+1} = 2^{k+1+1} - 1$$

which verifies $P(k+1)$ and completes the proof. ●

The following summation formula is the most important one because it occurs so often in the analysis of algorithms. If you don't remember anything else, you should remember this formula for the sum of the first n positive integers.

PRACTICE 7 Prove that for any positive integer n,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

■

Not all proofs by induction involve formulas with sums. Other algebraic identities about the positive integers, as well as nonalgebraic assertions like the number of offspring in generation n of the Smith family, can be proved by induction.

EXAMPLE 16 Prove that for any positive integer n, $2^n > n$.

$P(1)$ is the assertion $2^1 > 1$, which is surely true. Now we assume $P(k)$, $2^k > k$, and try to conclude $P(k+1)$, $2^{k+1} > k+1$. Now, where is $P(k)$ hidden in here? Aha—we can write the left side of $P(k+1)$, 2^{k+1}, as $2k \cdot 2$, and there's the left side of $P(k)$. Using the inductive hypothesis $2^k > k$ and multiplying both sides of this inequality by 2, we get $2^k \cdot 2 > k \cdot 2$. We complete the argument

$$2^{k+1} = 2^k \cdot 2 > k \cdot 2 = k + k \geq k + 1 \text{ (because } k \geq 1\text{)}$$

or

$$2^{k+1} > k + 1$$

●

For the first step of the induction process, it may be appropriate to begin at 0 or at 2 or 3 instead of at 1. The same principle applies, no matter where you first hop on the ladder.

EXAMPLE 17 Prove that $n^2 > 3n$ for $n \geq 4$.

Here we should use induction and begin with a basis step of $P(4)$. (Testing values of $n = 1, 2$, and 3 shows that the inequality does not hold for these values.) $P(4)$ is the inequality $4^2 > 3(4)$, or $16 > 12$, which is true. The inductive hypothesis is that $k^2 > 3k$ and that $k \geq 4$, and we want to show that $(k + 1)^2 > 3(k + 1)$.

$$\begin{aligned}
(k + 1)^2 &= k^2 + 2k + 1 \\
&> 3k + 2k + 1 && \text{(by the inductive hypothesis)} \\
&\geq 3k + 8 + 1 && \text{(because } k \geq 4\text{)} \\
&= 3k + 9 \\
&> 3k + 3 && \text{(because } 9 > 3\text{)} \\
&= 3(k + 1)
\end{aligned}$$

In this proof we used the fact that $3k + 9 > 3k + 3$. Of course, $3k + 9$ is greater than lots of things, but $3k + 3$ is what gives us what we want. In an induction proof, because we know exactly what we want as the result, we can let that guide us as we manipulate algebraic expressions. ●

PRACTICE 8 Prove that for all $n > 1$,

$$2^{n+1} < 3^n$$

■

EXAMPLE 18 Prove that for any positive integer n, the number $2^{2n} - 1$ is divisible by 3.

The basis step is to show $P(1)$, that $2^{2(1)} - 1 = 4 - 1 = 3$ is divisible by 3. Clearly this is true.

We assume that $2^{2k} - 1$ is divisible by 3, which means that $2^{2k} - 1 = 3m$ for some integer m, or $2^{2k} = 3m + 1$ (this little rewriting trick is the key to these "divisibility" problems). We want to show that $2^{2(k+1)} - 1$ is divisible by 3.

$$\begin{aligned}
2^{2(k+1)} - 1 &= 2^{2k+2} - 1 \\
&= 2^2 \cdot 2^{2k} - 1 \\
&= 2^2(3m + 1) - 1 && \text{(by the inductive hypothesis)} \\
&= 12m + 4 - 1 \\
&= 12m + 3 \\
&= 3(4m + 1) && \text{where } 4m + 1 \text{ is an integer}
\end{aligned}$$

Thus $2^{2(k+1)} - 1$ is divisible by 3. ●

Misleading claims of proof by induction are also possible. When we prove the truth of $P(k + 1)$ without relying on the truth of $P(k)$, we have done a direct proof of $P(k + 1)$ where $k + 1$ is arbitrary. The proof is not invalid, but it

should be rewritten to show that it is a direct proof of $P(n)$ for any n, not a proof by induction.

An inductive proof may be called for when its application is not as obvious as in the above examples. The problem statement may not directly say "prove something about nonnegative integers." Instead, there is some quantity in the statement to be proved that can take on arbitrary nonnegative integer values.

EXAMPLE 19

A programming language might be designed with the following convention regarding multiplication: A single factor requires no parentheses, but the product "a times b" must be written as $(a)b$. So the product

$$a \cdot b \cdot c \cdot d \cdot e \cdot f \cdot g$$

could be written in this language as

$$((((((a)b)c)d\,)e)\,f\,)g$$

or, for example,

$$((a)b)(((c)d\,)(e)\,f\,)g$$

depending on the order in which the products are formed. The result is the same in either case.

We want to show that any product of factors can be written with an even number of parentheses. The proof is by induction on the number of factors (this is where the nonnegative integer comes in—it represents the number of factors in any product of factors). For a single factor, there are 0 parentheses, an even number. Assume that for any product of k factors there is an even number of parentheses. Now consider a product P of $k + 1$ factors. P can be thought of as r times s where r has k factors and s is a single factor. By the inductive hypothesis, r has an even number of parentheses. Then we write r times s as $(r)s$. This adds 2 more parentheses to the even number of parentheses in r, giving P an even number of parentheses.

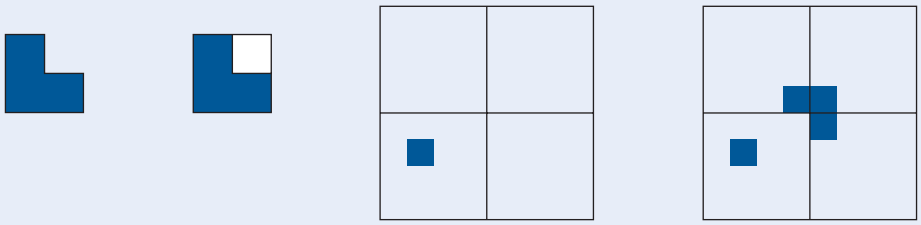
Here's an important observation about the proof in Example 19: There are no algebraic expressions! The entire proof is a verbal argument. For some proofs, we can't rely on just the crutch of nonverbal mathematical manipulations; we have to use words.

EXAMPLE 20

A "tiling" problem gives a nice illustration of induction in a geometric setting. An *angle iron* is an L-shaped piece that can cover 3 squares on a checkerboard (Figure 2.3a). The problem is to show that for any positive integer n, a $2^n \times 2^n$ checkerboard with one square removed can be tiled—completely covered—by angle irons.

The base case is $n = 1$, which gives a 2×2 checkerboard. Figure 2.3b shows the solution to this case if the upper right corner is removed. Removing any of the other three corners works the same way. Assume that any $2^k \times 2^k$ checkerboard with one square removed can be tiled using angle irons. Now consider a checkerboard with

dimensions $2^{k+1} \times 2^{k+1}$. We need to show that it can be tiled when one square is removed. To relate the $k + 1$ case to the inductive hypothesis, divide the $2^{k+1} \times 2^{k+1}$ checkerboard into four quarters. Each quarter will be a $2^k \times 2^k$ checkerboard, and one will have a missing square (Figure 2.3c). By the inductive hypothesis, this checkerboard can be tiled. Remove a corner from each of the other three checkerboards, as in Figure 2.3d. By the inductive hypothesis, the three boards with the holes removed can be tiled, and one angle iron can tile the three holes. Hence the original $2^{k+1} \times 2^{k+1}$ board with its one hole can be tiled.



(a) (b) (c) (d)

Figure 2.3

Second Principle of Induction

In addition to the first principle of induction, which we have been using,

$$\left.\begin{array}{l} 1.\ P(1) \text{ is true} \\ 2.\ (\forall k)[P(k) \text{ true} \rightarrow P(k+1) \text{ true}] \end{array}\right\} \rightarrow P(n) \text{ true for all positive integers } n$$

there is a second principle of induction.

PRINCIPLE **SECOND PRINCIPLE OF MATHEMATICAL INDUCTION**

$$\left.\begin{array}{l} 1'.\ P(1) \text{ is true} \\ 2'.\ (\forall k)[P(r) \text{ true for all } r, \\ \quad 1 \le r \le k \rightarrow P(k+1) \text{ true}] \end{array}\right\} \rightarrow P(n) \text{ true for all positive integers } n$$

These two induction principles differ in statements 2 and 2′. In statement 2, we must be able to prove for an arbitrary positive integer k that $P(k + 1)$ is true based only on the assumption that $P(k)$ is true. In statement 2′, we can assume that $P(r)$ is true for all integers r between 1 and an arbitrary positive integer k in order to prove that $P(k + 1)$ is true. This seems to give us a great deal more "ammunition," so we might sometimes be able to prove the implication in 2′ when we cannot prove the implication in 2.

What allows us to deduce $(\forall n)P(n)$ in either case? We will see that the two induction principles themselves, that is, the two methods of proof, are equivalent. In other words, if we accept the first principle of induction as valid, then the second principle of induction is valid, and conversely. In order to prove the equivalence of the two induction principles, we'll introduce another principle, which seems so obvious as to be unarguable.

PRINCIPLE **PRINCIPLE OF WELL-ORDERING**

Every collection of positive integers that contains any members at all has a smallest member.

We will see that the following implications are true:

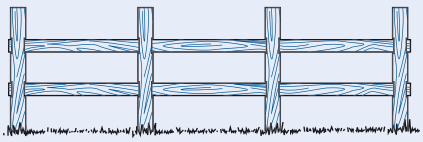
second principle of induction $\rightarrow$ first principle of induction
first principle of induction $\rightarrow$ well-ordering
well-ordering $\rightarrow$ second principle of induction

As a consequence, all three principles are equivalent, and accepting any one of them as true means accepting the other two as well.

To prove that the second principle of induction implies the first principle of induction, suppose we accept the second principle as valid reasoning. We then want to show that the first principle is valid; that is, that we can conclude $P(n)$ for all n from statements 1 and 2. If statement 1 is true, so is statement 1′. If statement 2 is true, then so is statement 2′, because we can say that we concluded $P(k+1)$ from $P(r)$ for all r between 1 and k, even though we used only the single condition $P(k)$. (More precisely, statement 2′ requires that we prove $P(1) \wedge P(2) \wedge \cdots \wedge P(k) \rightarrow P(k+1)$, but $P(1) \wedge P(2) \wedge \cdots \wedge P(k) \rightarrow P(k)$, and from statement 2, $P(k) \rightarrow P(k+1)$, so $P(1) \wedge P(2) \wedge \cdots \wedge P(k) \rightarrow P(k+1)$.) By the second principle of induction, we conclude $P(n)$ for all n. The proofs that the first principle of induction implies well-ordering and that well-ordering implies the second principle of induction are left as exercises in Section 4.1.

To distinguish between a proof by the first principle of induction and a proof by the second principle of induction, let's look at a rather picturesque example that can be proved both ways.

EXAMPLE 21 Prove that a straight fence with n fence posts has $n-1$ sections for any $n \geq 1$ (as in Figure 2.4a).

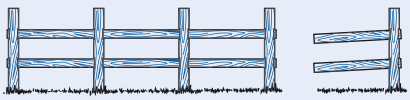


Fence with 4 fenceposts, 3 sections

(a)

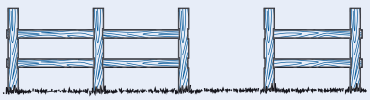
Fence with 1 fencepost, 0 sections

(b)



Fence with last post and last section removed

(c)



Fence with 1 section removed

(d)

Figure 2.4

Let $P(n)$ be the statement that a fence with n fence posts has $n - 1$ sections, and prove $P(n)$ true for all $n \geq 1$.

We'll start with the first principle of induction. For the basis step, $P(1)$ says that a fence with only 1 fence post has 0 sections, which is clearly true (Figure 2.4b). Assume that $P(k)$ is true:

$$\text{a fence with } k \text{ fence posts has } k - 1 \text{ sections}$$

and try to prove $P(k + 1)$:

$$\text{(?) a fence with } k + 1 \text{ fence posts has } k \text{ sections}$$

Given a fence with $k + 1$ fence posts, how can we relate that to a fence with k fence posts so that we can make use of the inductive hypothesis? We can chop off the last post and the last section (Figure 2.4c). The remaining fence has k fence posts and, by the inductive hypothesis, $k - 1$ sections. Therefore the original fence had k sections.

Now we'll prove the same result using the second principle of induction. The basis step is the same as before. For the inductive hypothesis, we assume

$$\text{for all } r,\ 1 \leq r \leq k, \text{ a fence with } r \text{ fence posts has } r - 1 \text{ sections}$$

and try to prove $P(k + 1)$:

$$\text{(?) a fence with } k + 1 \text{ fence posts has } k \text{ sections}$$

For a fence with $k + 1$ fence posts, split the fence into two parts by removing one section (Figure 2.4d). The 2 parts of the fence have r_1 and r_2 fence posts, where $1 \leq r_1 \leq k$, $1 \leq r_2 \leq k$, and $r_1 + r_2 = k + 1$. By the inductive hypothesis, the 2 parts have, respectively, $r_1 - 1$ and $r_2 - 1$ sections, so the original fence has

$$(r_1 - 1) + (r_2 - 1) + 1 \text{ sections}$$

(The extra 1 is for the one that we removed.) Simple arithmetic then yields

$$r_1 + r_2 - 1 = (k + 1) - 1 = k \text{ sections}$$

This proves that a fence with $k + 1$ fence posts has k sections, which verifies $P(k + 1)$ and completes the proof using the second principle of induction. ●

Example 21 allowed for either form of inductive proof because we could either reduce the fence at one end or split it at an arbitrary point. The problem of Example 19 is similar.

EXAMPLE 22

We again want to show that any product of factors can be written in this programming language with an even number of parentheses, this time using the second principle of induction. The base case is the same as in Example 19: A single factor has 0 parentheses, an even number. Assume that any product of r factors, $1 \leq r \leq k$, can be written with an even number of parentheses. Then consider a product

P with $k + 1$ factors. P can be written as $(S)T$, a product of two factors S and T, where S has r_1 factors and T has r_2 factors. Then $1 \leq r_1 \leq k$ and $1 \leq r_2 \leq k$, with $r_1 + r_2 = k + 1$. By the inductive hypothesis, S and T each have an even number of parentheses, and therefore so does $(S)T = P$. ●

Most problems do not work equally well with either form of induction; the fence post and the programming language problem were somewhat artificial. Generally, the second principle of induction is called for when the problem "splits" most naturally in the middle instead of growing from the end.

EXAMPLE 23 Prove that for every integer $n \geq 2$, n is a prime number or a product of prime numbers.

We will postpone the decision of whether to use the first or the second principle of induction; the basis step is the same in each case and need not start with 1. Obviously here we should start with 2. $P(2)$ is the statement that 2 is a prime number or a product of primes. Because 2 is a prime number, $P(2)$ is true. Jumping ahead, for either principle we will be considering the number $k + 1$. If $k + 1$ is prime, we are done. If $k + 1$ is not prime, then it is a composite number and can be written as $k + 1 = ab$. Here $k + 1$ has been split into two factors. Maybe neither of these factors has the value k, so an assumption only about $P(k)$ isn't enough. Hence, we'll use the second principle of induction.

So let's start again. We assume that for all r, $2 \leq r \leq k$, $P(r)$ is true—r is prime or the product of primes. Now consider the number $k + 1$. If $k + 1$ is prime, we are done. If $k + 1$ is not prime, then it is a composite number and can be written as $k + 1 = ab$, where $1 < a < k + 1$ and $1 < b < k + 1$. (This is a nontrivial factorization, so neither factor can be 1 or $k + 1$.) Therefore $2 \leq a \leq k$ and $2 \leq b \leq k$. The inductive hypothesis applies to both a and b, so a and b are either prime or the product of primes. Thus, $k + 1 = ab$ is the product of prime numbers. This verifies $P(k + 1)$ and completes the proof by the second principle of induction. ●

The proof in Example 23 is an *existence* proof rather than a *constructive* proof. Knowing that every nonprime number has a factorization as a product of primes does not make it easy to *find* such a factorization. (We will see in Section 2.4 that there is, except for the order of the factors, only one such factorization.) Some encryption systems for passing information in a secure fashion on the Web depend on the difficulty of factoring large numbers into their prime factors (see the discussion on public-key encryption in Section 5.6).

EXAMPLE 24 Prove that any amount of postage greater than or equal to 8 cents can be built using only 3-cent and 5-cent stamps.

Here we let $P(n)$ be the statement that only 3-cent and 5-cent stamps are needed to build n cents worth of postage, and prove that $P(n)$ is true for all $n \geq 8$. The basis step is to establish $P(8)$, which is done by the equation

$$8 = 3 + 5$$

For reasons that will be clear momentarily, we'll also establish two additional cases, $P(9)$ and $P(10)$, by the equations

$$9 = 3 + 3 + 3$$
$$10 = 5 + 5$$

Now we assume that $P(r)$ is true, that is, r can be written as a sum of $3s$ and $5s$, for any r, $8 \le r \le k$, and consider $P(k + 1)$. We may assume that $k + 1$ is at least 11, because we have already proved $P(r)$ true for $r = 8$, 9, and 10. If $k + 1 \ge 11$, then $(k + 1) - 3 = k - 2 \ge 8$. Thus $k - 2$ is a legitimate r value, and by the inductive hypothesis, $P(k - 2)$ is true. Therefore $k - 2$ can be written as a sum of $3s$ and $5s$, and adding an additional 3 gives us $k + 1$ as a sum of $3s$ and $5s$. This verifies that $P(k + 1)$ is true and completes the proof. ●

PRACTICE 9

a. Why are the additional cases $P(9)$ and $P(10)$ proved separately in Example 24?
b. Why can't the first principle of induction be used in the proof of Example 24? ■

REMINDER

Use the second principle of induction when the $k + 1$ case depends on results farther back than k.

As a general rule, the first principle of mathematical induction applies when information about "one position back" is enough, that is, when the truth of $P(k)$ is enough to prove the truth of $P(k + 1)$. The second principle applies when information about "one position back" isn't good enough; that is, you can't prove that $P(k + 1)$ is true just because you know $P(k)$ is true, but you can prove $P(k + 1)$ true if you know that $P(r)$ is true for one or more values of r that are "farther back" than k.

SECTION 2.2 REVIEW

TECHNIQUES

- Use the first principle of induction in proofs.
- Use the second principle of induction in proofs.

MAIN IDEAS

- Mathematical induction is a technique to prove properties of positive integers.
- An inductive proof need not begin with 1.
- Induction can be used to prove statements about quantities whose values are arbitrary nonnegative integers.
- The first and second principles of induction each prove the same conclusion, but one approach may be easier to use than the other in a given situation.

EXERCISES 2.2

1. For all positive integers, let $P(n)$ be the equation

$$2 + 6 + 10 + \cdots + (4n - 2) = 2n^2$$

a. Write the equation for the base case $P(1)$ and verify that it is true.
b. Write the inductive hypothesis $P(k)$.
c. Write the equation for $P(k + 1)$.
d. Prove that $P(k + 1)$ is true.

2. For all positive integers, let $P(n)$ be the equation

$$2 + 4 + 6 + \cdots + 2n = n(n + 1)$$

   a. Write the equation for the base case $P(1)$ and verify that it is true.
   b. Write the inductive hypothesis $P(k)$.
   c. Write the equation for $P(k + 1)$.
   d. Prove that $P(k + 1)$ is true.

In Exercises 3–26, use mathematical induction to prove that the statements are true for every positive integer n. [*Hint*: In the algebra part of the proof, if the final expression you want has factors and you can pull those factors out early, do that instead of multiplying everything out and getting some humongous expression.]

3. $1 + 5 + 9 + \cdots + (4n - 3) = n(2n - 1)$
4. $1 + 3 + 6 + \cdots + \dfrac{n(n + 1)}{2} = \dfrac{n(n + 1)(n + 2)}{6}$
5. $4 + 10 + 16 + \cdots + (6n - 2) = n(3n + 1)$
6. $5 + 10 + 15 + \cdots + 5n = \dfrac{5n(n + 1)}{2}$
7. $1^2 + 2^2 + \cdots + n^2 = \dfrac{n(n + 1)(2n + 1)}{6}$
8. $1^3 + 2^3 + \cdots + n^3 = \dfrac{n^2(n + 1)^2}{4}$
9. $1^2 + 3^2 + \cdots + (2n - 1)^2 = \dfrac{n(2n - 1)(2n + 1)}{3}$
10. $1^4 + 2^4 + \cdots + n^4 = \dfrac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$
11. $1 \cdot 3 + 2 \cdot 4 + 3 \cdot 5 + \cdots + n(n + 2) = \dfrac{n(n + 1)(2n + 7)}{6}$
12. $1 + a + a^2 + \cdots + a^{n-1} = \dfrac{a^n - 1}{a - 1}$ for $a \neq 0, a \neq 1$
13. $\dfrac{1}{1 \cdot 2} + \dfrac{1}{2 \cdot 3} + \dfrac{1}{3 \cdot 4} + \cdots + \dfrac{1}{n(n + 1)} = \dfrac{n}{n + 1}$
14. $\dfrac{1}{1 \cdot 3} + \dfrac{1}{3 \cdot 5} + \dfrac{1}{5 \cdot 7} + \cdots + \dfrac{1}{(2n - 1)(2n + 1)} = \dfrac{n}{2n + 1}$
15. $1^2 - 2^2 + 3^2 - 4^2 + \cdots + (-1)^{n+1}n^2 = \dfrac{(-1)^{n+1}(n)(n + 1)}{2}$
16. $2 + 6 + 18 + \cdots + 2 \cdot 3^{n-1} = 3^n - 1$
17. $2^2 + 4^2 + \cdots + (2n)^2 = \dfrac{2n(n + 1)(2n + 1)}{3}$
18. $1 \cdot 2^1 + 2 \cdot 2^2 + 3 \cdot 2^3 + \cdots + n \cdot 2^n = (n - 1)2^{n+1} + 2$
19. $1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + \cdots + n(n + 1) = \dfrac{n(n + 1)(n + 2)}{3}$

20. $1 \cdot 2 \cdot 3 + 2 \cdot 3 \cdot 4 + \cdots + n(n+1)(n+2) = \dfrac{n(n+1)(n+2)(n+3)}{4}$

21. $\dfrac{1}{1 \cdot 4} + \dfrac{1}{4 \cdot 7} + \dfrac{1}{7 \cdot 10} + \cdots + \dfrac{1}{(3n-2)(3n+1)} = \dfrac{n}{3n+1}$

22. $1 \cdot 1! + 2 \cdot 2! + 3 \cdot 3! + \cdots + n \cdot n! = (n+1)! - 1$ where $n!$ is the product of the positive integers from 1 to n.

23. $1 + 4 + 4^2 + \cdots + 4^n = \dfrac{4^{n+1} - 1}{3}$

24. $1 + x + x^2 + \cdots + x^n = \dfrac{x^{n+1} - 1}{x - 1}$ where x is any integer > 1

25. $1 + 4 + 7 + 10 + \cdots + (3n-2) = \dfrac{n(3n-1)}{2}$

26. $1 + [x \cdot 2 - (x-1)] + [x \cdot 3 - (x-1)] + \cdots + [x \cdot n - (x-1)] = \dfrac{n[xn - (x-2)]}{2}$ where x is any integer ≥ 1

27. A *geometric progression* (*geometric sequence*) is a sequence of terms where there is an initial term a and each succeeding term is obtained by multiplying the previous term by a *common ratio* r. Prove the formula for the sum of the first n ($n \geq 1$) terms of a geometric sequence where $r \neq 1$:

$$a + ar + ar^2 + \cdots + ar^{n-1} = \frac{a - ar^n}{1 - r}$$

28. An *arithmetic progression* (*arithmetic sequence*) is a sequence of terms where there is an initial term a and each succeeding term is obtained by adding a *common difference* d to the previous term. Prove the formula for the sum of the first n ($n \geq 1$) terms of an arithmetic sequence:

$$a + (a+d) + (a+2d) + \cdots + [a + (n-1)d] = \frac{n}{2}[2a + (n-1)d]$$

29. Using Exercises 27 and 28, find an expression for the value of the following sums.
    a. $2 + 2 \cdot 5 + 2 \cdot 5^2 + \cdots + 2 \cdot 5^9$
    b. $4 \cdot 7 + 4 \cdot 7^2 + 4 \cdot 7^3 + \cdots + 4 \cdot 7^{12}$
    c. $1 + 7 + 13 + \cdots + 49$
    d. $12 + 17 + 22 + 27 + \cdots + 92$

30. Prove that

$$(-2)^0 + (-2)^1 + (-2)^2 + \cdots + (-2)^n = \frac{1 - 2^{n+1}}{3}$$

    for every positive odd integer n.

31. Prove that $n^2 > n + 1$ for $n \geq 2$.

32. Prove that $n^2 \geq 2n + 3$ for $n \geq 3$.

33. Prove that $n^2 > 5n + 10$ for $n > 6$.

34. Prove that $2^n > n^2$ for $n \geq 5$.

In Exercises 35–40, $n!$ is the product of the positive integers from 1 to n.

35. Prove that $n! > n^2$ for $n \geq 4$.

36. Prove that $n! > n^3$ for $n \geq 6$.
37. Prove that $n! > 2^n$ for $n \geq 4$.
38. Prove that $n! > 3^n$ for $n \geq 7$.
39. Prove that $n! \geq 2^{n-1}$ for $n \geq 1$.
40. Prove that $n! < n^n$ for $n \geq 2$.
41. Prove that $(1 + x)^n > 1 + x^n$ for $n > 1, x > 0$.
42. Prove that $\left(\frac{a}{b}\right)^{n+1} < \left(\frac{a}{b}\right)^n$ for $n \geq 1$ and $0 < a < b$.
43. Prove that $1 + 2 + \cdots + n < n^2$ for $n > 1$.
44. Prove that $1 + \frac{1}{4} + \frac{1}{9} + \cdots + \frac{1}{n^2} < 2 - \frac{1}{n}$ for $n \geq 2$
45. a. Try to use induction to prove that

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} < 2 \qquad \text{for } n \geq 1$$

What goes wrong?

b. Prove that

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} = 2 - \frac{1}{2^n} \qquad \text{for } n \geq 1$$

thus showing that

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} < 2 \qquad \text{for } n \geq 1$$

46. Prove that

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{2^n} \geq 1 + \frac{n}{2} \qquad \text{for } n \geq 1$$

(Note that the denominators increase by 1, not by powers of 2.)

For Exercises 47–58, prove that the statements are true for every positive integer.

47. $2^{3n} - 1$ is divisible by 7.
48. $3^{2n} + 7$ is divisible by 8.
49. $7^n - 2^n$ is divisible by 5.
50. $13^n - 6^n$ is divisible by 7.
51. $2^n + (-1)^{n+1}$ is divisible by 3.
52. $2^{5n+1} + 5^{n+2}$ is divisible by 27.
53. $3^{4n+2} + 5^{2n+1}$ is divisible by 14.
54. $7^{2n} + 16n - 1$ is divisible by 64.
55. $10^n + 3 \cdot 4^{n+2} + 5$ is divisible by 9.
56. $n^3 - n$ is divisible by 3.
57. $n^3 + 2n$ is divisible by 3.
58. $x^n - 1$ is divisible by $x - 1$ for $x \neq 1$.

59. Prove DeMoivre's Theorem:

$$(\cos\theta + i\sin\theta)^n = \cos n\theta + i\sin n\theta$$

for all $n \geq 1$. *Hint:* Recall the addition formulas from trigonometry:

$$\cos(\alpha + \beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$
$$\sin(\alpha + \beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$$

60. Prove that

$$\sin\theta + \sin 3\theta + \cdots + \sin(2n-1)\theta = \frac{\sin^2 n\theta}{\sin\theta}$$

for all $n \geq 1$ and all θ for which $\sin\theta \neq 0$.

61. Use induction to prove that the product of any three consecutive positive integers is divisible by 3.

62. Suppose that exponentiation is defined by the equation

$$x^j \cdot x = x^{j+1}$$

for any $j \geq 1$. Use induction to prove that $x^n \cdot x^m = x^{n+m}$ for $n \geq 1$, $m \geq 1$.
(*Hint:* Do induction on m for a fixed, arbitrary value of n.)

63. According to Example 20, it is possible to use angle irons to tile a 4×4 checkerboard with the upper right corner removed.Sketch such a tiling.

64. Example 20 does not cover the case of checkerboards that are not sized by powers of 2. Determine whether it is possible to tile a 3×3 checkerboard.

65. Prove that it is possible to use angle irons to tile a 5×5 checkerboard with the upper left corner removed.

66. Find a configuration for a 5×5 checkerboard with one square removed that is not possible to tile; explain why this is not possible.

67. Consider n infinitely long straight lines, none of which are parallel and no three of which have a common point of intersection. Show that for $n \geq 1$, the lines divide the plane into $(n^2 + n + 2)/2$ separate regions.

68. A string of 0*s* and 1*s* is to be processed and converted to an even-parity string by adding a parity bit to the end of the string.(For an explanation of the use of parity bits, see Example 30 in Chapter 9.) The parity bit is initially 0. When a 0 character is processed, the parity bit remains unchanged. When a 1 character is processed, the parity bit is switched from 0 to 1 or from 1 to 0. Prove that the number of 1*s* in the final string, that is, including the parity bit, is always even. (*Hint:* Consider various cases.)

69. What is wrong with the following "proof" by mathematical induction? We will prove that for any positive integer n, n is equal to 1 more than n. Assume that $P(k)$ is true.

$$k = k + 1$$

Adding 1 to both sides of this equation, we get

$$k + 1 = k + 2$$

Thus,

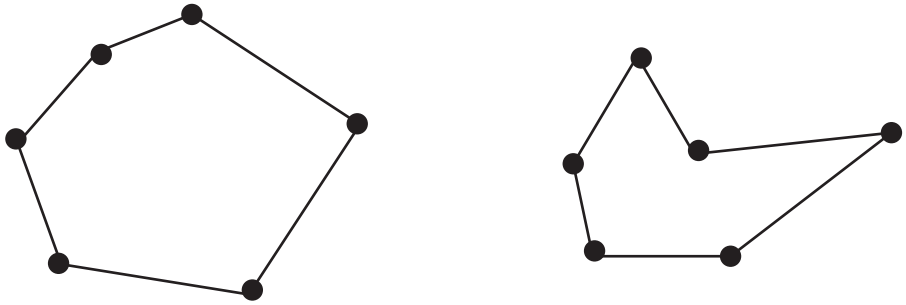
$$P(k+1) \text{ is true}$$

70. What is wrong with the following "proof" by mathematical induction?

> We will prove that all computers are built by the same manufacturer. In particular, we will prove that in any collection of n computers where n is a positive integer, all the computers are built by the same manufacturer. We first prove $P(1)$, a trivial process, because in any collection consisting of

one computer, there is only one manufacturer. Now we assume $P(k)$; that is, in any collection of k computers, all the computers were built by the same manufacturer. To prove $P(k + 1)$, we consider any collection of $k + 1$ computers. Pull one of these $k + 1$ computers (call it HAL) out of the collection. By our assumption, the remaining k computers all have the same manufacturer. Let HAL change places with one of these k computers. In the new group of k computers, all have the same manufacturer. Thus, HAL's manufacturer is the same one that produced all the other computers, and all $k + 1$ computers have the same manufacturer.

71. An obscure tribe has only three words in its language, *moon*, *noon*, and *soon*. New words are composed by juxtaposing these words in any order, as in *soonnoonmoonnoon*. Any such juxtaposition is a legal word.
    a. Use the first principle of induction (on the number of subwords in the word) to prove that any word in this language has an even number of o's.
    b. Use the second principle of induction (on the number of subwords in the word) to prove that any word in this language has an even number of o's.
72. A *simple closed polygon* consists of n points in the plane joined in pairs by n line segments; each point is the endpoint of exactly 2 line segments. Following are two examples.



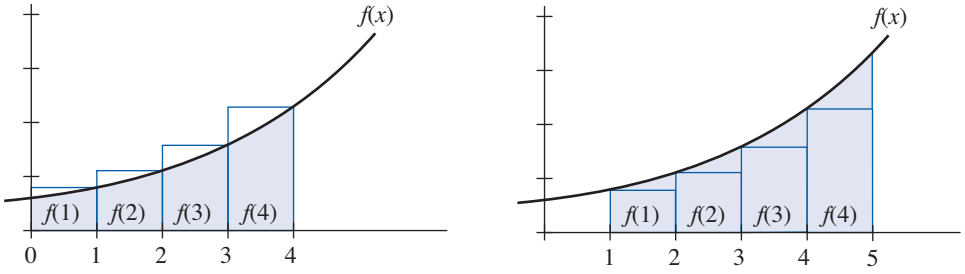
    a. Use the first principle of induction to prove that the sum of the interior angles of an n-sided simple closed polygon is $(n - 2)180°$ for all $n \geq 3$.
    b. Use the second principle of induction to prove that the sum of the interior angles of an n-sided simple closed polygon is $(n - 2)180°$ for all $n \geq 3$.
73. The Computer Science club is sponsoring a jigsaw puzzle contest. Jigsaw puzzles are assembled by fitting 2 pieces together to form a small block, adding a single piece to a block to form a bigger block, or fitting 2 blocks together. Each of these moves is considered a step in the solution. Use the second principle of induction to prove that the number of steps required to assemble an n-piece jigsaw puzzle is $n - 1$.
74. OurWay Pizza makes only two kinds of pizza, pepperoni and vegetarian. Any pizza of either kind comes with an even number of breadsticks (not necessarily the same even number for both kinds). Any order of 2 or more pizzas must include at least 1 of each kind. When the delivery driver goes to deliver an order, he or she puts the completed order together by combining 2 suborders—picking up all the pepperoni pizzas from 1 window and all the vegetarian pizzas from another window. Prove that for a delivery of n pizzas, $n \geq 1$, there are an even number of breadsticks included.
75. Consider propositional wffs that contain only the connectives $\wedge$, $\vee$, and $\rightarrow$ (no negation) and where wffs must be parenthesized when joined by a logical connective. Count each statement letter, connective, or parenthesis as one symbol. For example, $((A) \wedge (B)) \vee ((C) \wedge (D))$ is such a wff, with 19 symbols. Prove that any such wff has an odd number of symbols.
76. In any group of k people, $k \geq 1$, each person is to shake hands with every other person. Find a formula for the number of handshakes, and prove the formula using induction.

77. Prove that any amount of postage greater than or equal to 2 cents can be built using only 2-cent and 3-cent stamps.
78. Prove that any amount of postage greater than or equal to 12 cents can be built using only 4-cent and 5-cent stamps.
79. Prove that any amount of postage greater than or equal to 14 cents can be built using only 3-cent and 8-cent stamps.
80. Prove that any amount of postage greater than or equal to 42 cents can be built using only 4-cent and 15-cent stamps.
81. Prove that any amount of postage greater than or equal to 64 cents can be built using only 5-cent and 17-cent stamps.
82. Your bank ATM delivers cash using only \$20 and \$50 bills. Prove that you can collect, in addition to \$20, any multiple of \$10 that is \$40 or greater.

Exercises 83–84 require familiarity with ideas from calculus. Exercises 1–26 give exact formulas for the sum of terms in a sequence that can be expressed as $\sum_{m=1}^{n} f(m)$. Sometimes it is difficult to find an exact expression for this summation, but if the value of $f(m)$ increases monotonically, integration can be used to find upper and lower bounds on the value of the summation. Specifically,

$$\int_0^n f(x)dx \le \sum_{m=1}^{n} f(m) \le \int_1^{n+1} f(x)dx$$

Using the following figure, we can see (on the left) that $\int_0^n f(x)dx$ underestimates the value of the summation while (on the right) $\int_1^{n+1} f(x)dx$ overestimates it.



83. Show that $\int_0^n 2x\,dx \le \sum_{m=1}^{n} 2m \le \int_1^{n+1} 2x\,dx$ (see Exercise 2).

84. Show that $\int_0^n x^2dx \le \sum_{m=1}^{n} m^2 \le \int_1^{n+1} x^2dx$ (see Exercise 7).

SECTION 2.3 MORE ON PROOF OF CORRECTNESS

In Section 1.6, we explained the use of a formal logic system to prove mathematically the correctness of a program. Assertions or predicates involving the program variables are inserted at the beginning, at the end, and at intermediate points between the program statements. Then proving the correctness of any particular program statement s_i involves proving that the implication represented by the Hoare triple

$$\{Q\}\ s_i\ \{R\} \tag{1}$$

is true.Here Q and R are assertions known, respectively, as the precondition and postcondition for the statement. The program is provably correct if all such implications for the statements in the program are true.

In Chapter 1, we discussed rules of inference that give conditions under which implication (1) is true when s_i is an assignment statement and when s_i is a conditional statement. Now we will use a rule of inference that gives conditions under which implication (1) is true when s_i is a loop statement. We have deferred consideration of loop statements until now because mathematical induction is used in applying this rule of inference.

Loop Rule

Suppose that s_i is a loop statement in the form

while condition B **do**
  P
end while

where B is a condition that is either true or false and P is a program segment. When this statement is executed, condition B is evaluated. If B is true, program segment P is executed and then B is evaluated again. If B is still true, program segment P is executed again, then B is evaluated again, and so forth. If condition B ever evaluates to false, the loop terminates.

The form of implication (1) that can be used when s_i is a loop statement imposes (like the assignment rule did) a relationship between the precondition and the postcondition. The precondition Q holds before the loop is entered; strangely enough, one requirement is that Q must continue to hold after the loop terminates (which means that we should look for a Q that we want to be true when the loop terminates). In addition, B'—the condition for loop termination—must be true then as well. Thus (1) will have the form

$$\{Q\}\ s_i\ \{Q \wedge B'\} \tag{2}$$

EXAMPLE 25 Consider the following pseudocode function, which is supposed to return the value $x * y$ for nonnegative integers x and y.

```
Product (nonnegative integer x; nonnegative integer y)
Local variables:
integers i, j
   i = 0
   j = 0
   while i ≠ x do
      j = j + y
      i = i + 1
   end while
   //j now has the value x * y
   return j
end function Product
```

This function contains a loop; the condition B for continued loop execution is $i \neq x$. The condition B' for loop termination is therefore $i = x$. When the loop terminates, it is claimed in the comment that j has the value $x * y$. Thus, on loop termination, we want

$$Q \wedge B' = Q \wedge (i = x)$$

and we also want

$$j = x * y$$

To have both

$$Q \wedge (i = x) \text{ and } j = x * y$$

Q must be the assertion

$$j = i * y$$

(Notice that Q is a predicate, that is, it states a relationship between variables in the program. It is never part of an equation such as $Q = j$.) To match the form of (2), the assertion $j = i * y$ would have to be true before the loop statement. This is indeed the case because right before the loop statement, $i = j = 0$.

It would seem that for this example we have a candidate assertion Q for implication (2), but we do not yet have the rule of inference that allows us to say when (2) is a true implication. (Remember that we discovered our Q by "wishful thinking" about the correct operation of the function code.)

Assertion Q must be true before the loop is entered. If implication (2) is to hold, Q must remain true after the loop terminates. Because it may not be known exactly when the loop will terminate, Q must remain true after each iteration through the loop, which will include the final iteration. Q represents a predicate, or relation, among the values of the program variables. If this relation holds among the values of the program variables before a loop iteration executes and holds among the values after the iteration executes, then the *relation* among these variables is unaffected by the action of the loop iteration, even though the values themselves may be changed. Such a relation is called a **loop invariant**.

The **loop rule of inference** allows the truth of (2) to be inferred from an implication stating that Q is a loop invariant. Again, for Q to be a loop invariant it must be the case that if Q is true and condition B is true, so that another loop iteration is executed, then Q remains true after that iteration, which can be expressed by the Hoare triple $\{Q \wedge B\}\ P\ \{Q\}$. The rule is formally stated in Table 2.4.

TABLE 2.4

From	Can Derive	Name of Rule	Restrictions on Use

$\{Q \wedge B\}\ P\ \{Q\}$	$\{Q\}\ s_i\ \{Q \wedge B'\}$	loop	s_i has the form
			while condition B **do**
			P
			end while

To use this rule of inference, we must find a *useful* loop invariant Q—one that asserts what we want and expect to have happen—and then prove the implication

$$\{Q \wedge B\}\ P\ \{Q\}$$

Here is where induction comes into play. We denote by $Q(n)$ the statement that a proposed loop invariant Q is true after n iterations of the loop. Because we do not necessarily know how many iterations the loop may execute (that is, how long condition B remains true), we want to show that $Q(n)$ is true for all $n \geq 0$. (The value of $n = 0$ corresponds to the assertion upon entering the loop, after zero loop iterations.)

EXAMPLE 26 Consider again the pseudocode function of Example 25. In that example, we guessed that Q is the relation

$$j = i * y$$

To use the loop rule of inference, we must prove that Q is a loop invariant.

The quantities x and y remain unchanged throughout the function, but values of i and j change within the loop. We let i_n and j_n denote the values of i and j, respectively, after n iterations of the loop. Then $Q(n)$ is the statement $j_n = i_n * y$.

We prove by induction that $Q(n)$ holds for all $n \geq 0$. $Q(0)$ is the statement

$$j_0 = i_0 * y$$

which, as we noted in Example 25, is true, because after zero iterations of the loop, when we first get to the loop statement, both i and j have been assigned the value 0. (Formally, the assignment rule could be used to prove that these conditions on i and j hold at this point.)

Assume $Q(k)$: $j_k = i_k * y$
Show $Q(k + 1)$: $j_{k+1} = i_{k+1} * y$

Between the time j and i have the values j_k and i_k and the time they have the values j_{k+1} and i_{k+1}, one iteration of the loop takes place. In that iteration, j is changed by adding y to the previous value, and i is changed by adding 1. Thus,

$$j_{k+1} = j_k + y \quad (3)$$
$$i_{k+1} = i_k + 1 \quad (4)$$

Then

$$\begin{aligned} j_{k+1} &= j_k + y && \text{(by (3))} \\ &= i_k * y + y && \text{(by the inductive hypothesis)} \\ &= (i_k + 1)y \\ &= i_{k+1} * y && \text{(by (4))} \end{aligned}$$

We have proved that Q is a loop invariant.

The loop rule of inference allows us to infer that after the loop statement is exited, the condition $Q \wedge B'$ holds, which in this case becomes

$$j = i * y \wedge i = x$$

Therefore at this point the statement

$$j = x * y$$

is true, which is exactly what the function is intended to compute. ■

Example 26 illustrates that loop invariants say something stronger about the program than we actually want to show; what we want to show is the special case of the loop invariant on termination of the loop. Finding the appropriate loop invariant requires working backward from the desired conclusion, as in Example 25.

We did not, in fact, prove that the loop in this example actually does terminate. What we proved was **partial correctness**—the program produces the correct answer, given that execution does terminate. Because x is a nonnegative integer and i is an integer that starts at 0 and is then incremented by 1 at each pass through the loop, we know that eventually $i = x$ will become true.

PRACTICE 10 Show that the following function returns the value $x + y$ for nonnegative integers x and y by proving the loop invariant Q: $j = x + i$ and evaluating Q when the loop terminates.

```
Sum (nonnegative integer x; nonnegative integer y)
Local variables:
integers i, j
   i = 0
   j = x
   while i ≠ y do
      j = j + 1
      i = i + 1
   end while
   // j now has the value x + y
   return j
end function Sum
```

■

The two functions of Example 25 and Practice 10 are somewhat unrealistic; after all, if we wanted to compute $x * y$ or $x + y$, we could no doubt do it with a single program statement. However, the same techniques apply to more meaningful computations, such as the Euclidean algorithm.

Euclidean Algorithm

The **Euclidean algorithm** was described by the Greek mathematician Euclid over 2300 years ago, although it may have been known even earlier. At any rate, it is one of the oldest known algorithms.This algorithm finds the greatest common divisor of two positive integers a and b with $a > b$. The **greatest common divisor** of a and b, denoted by gcd(a, b), is the largest integer n such that $n \mid a$ and $n \mid b$. For example, gcd(12, 18) is 6 and gcd(420, 66) = 6.

First, let's dispense with two trivial cases of the gcd(a, b) that do not require the Euclidean algorithm.

i. gcd(a, a) = a. Clearly $a \mid a$ and no larger integer divides a.
ii. gcd(a, 0) = a. Again, $a \mid a$ and no larger integer divides a, but also $a \mid 0$ because 0 is a multiple of a: $0 = 0(a)$

The Euclidean algorithm works by a succession of divisions. To find gcd(a, b), assuming that $a > b$, you first divide a by b, getting a quotient and a remainder. More formally, at this point $a = q_1 b + r_1$, where $0 \leq r_1 < b$. Next you divide the divisor, b, by the remainder, r_1, getting $b = q_2 r_1 + r_2$, where $0 \leq r_2 < r_1$. Again divide the divisor, r_1, by the remainder, r_2, getting $r_1 = q_3 r_2 + r_3$, where $0 \leq r_3 < r_2$. Clearly, there is a looping process going on, with the remainders getting successively smaller. The process terminates when the remainder is 0, at which point the greatest common divisor is the last divisor used.

EXAMPLE 27 To find gcd(420, 66) the following divisions are performed:

$$\begin{array}{r} 6 \\ 66\overline{)420} \\ \underline{396} \\ 24 \end{array} \qquad \begin{array}{r} 2 \\ 24\overline{)66} \\ \underline{48} \\ 18 \end{array} \qquad \begin{array}{r} 1 \\ 18\overline{)24} \\ \underline{18} \\ 6 \end{array} \qquad \begin{array}{r} 3 \\ 6\overline{)18} \\ \underline{18} \\ 0 \end{array}$$

The answer is 6, the divisor used when the remainder became 0. ●

A pseudocode version of the algorithm follows, given in the form of a function to return gcd(a, b).

ALGORITHM ***EUCLIDEAN ALGORITHM***

GCD (positive integer a; positive integer b)
//$a > b$
Local variables:
integers i, j
  $i = a$
  $j = b$

```
    while j ≠ 0 do
        compute i = qj + r, 0 ≤ r < j
        i = j
        j = r
    end while
    // i now has the value gcd(a, b)
    return i;
end function GCD
```

We intend to prove the correctness of this function, but we will need one additional fact first, namely,

$$(\forall \text{ integers } a, b, q, r)[(a = qb + r) \rightarrow (\gcd(a, b) = \gcd(b, r))] \tag{5}$$

To prove (5), assume that a $= qb + r$ and suppose that c divides both a and b so that $a = q_1c$ and $b = q_2c$. Then

$$r = a - qb = q_1c - qq_2c = c(q_1 - qq_2)$$

so that c divides r as well. Therefore anything that divides a and b also divides b and r. Now suppose d divides both b and r so that $b = q_3d$ and $r = q_4d$. Then

$$a = qb + r = qq_3d + q_4d = d(qq_3 + q_4)$$

so that d divides a as well. Therefore anything that divides b and r also divides a and b. Because (a, b) and (b, r) have identical divisors, they must have the same greatest common divisor.

EXAMPLE 28 Prove the correctness of the Euclidean algorithm.

Using function GCD, we will prove the loop invariant Q: $\gcd(i, j) = \gcd(a, b)$ and evaluate Q when the loop terminates. We use induction to prove $Q(n)$: $\gcd(i_n, j_n) = \gcd(a, b)$ for all $n \geq 0$. $Q(0)$ is the statement

$$\gcd(i_0, j_0) = \gcd(a, b)$$

which is true because when we first get to the loop statement, i and j have the values a and b, respectively.

Assume $Q(k)$: $\gcd(i_k, j_k) = \gcd(a, b)$
Show $Q(k + 1)$: $\gcd(i_{k+1}, j_{k+1}) = \gcd(a, b)$

By the assignment statements within the loop body, we know that

$$i_{k+1} = j_k$$
$$j_{k+1} = r_k$$

Then

$$\begin{aligned} \gcd(i_{k+1}, j_{k+1}) &= \gcd(j_k, r_k) \\ &= \gcd(i_k, j_k) && \text{by (5)} \\ &= \gcd(a, b) && \text{by the inductive hypothesis} \end{aligned}$$

Q is therefore a loop invariant. At loop termination, $\gcd(i, j) = \gcd(a, b)$ and $j = 0$, so $\gcd(i, 0) = \gcd(a, b)$. But $\gcd(i, 0)$ is i, so $i = \gcd(a, b)$. Therefore function GCD is correct. ●

Making Safer Software

Proof of correctness seeks to verify that a given computer program or segment of a program meets its specifications. As we have seen, this approach relies on formal logic to prove that if a certain relationship (the precondition) holds among the program variables before a given statement is executed, then after execution another relationship (the postcondition) holds. Because of the labor-intensive nature of proof of correctness, its use is typically reserved for critical sections of code in important applications.

The B method is a set of tools that does two things:

1. Supports formal project specification by means of an abstract model of the system to be developed. This support includes both automatic generation of lemmas that must be proven in order to guarantee that the model reflects the system requirements and automatic proof tools to prove each lemma or flag it for human verification assistance.
2. Translates the abstract model into a code-ready design, again using lemmas to ensure that the design matches the abstract model. The final level can then be translated into code, often using the Ada programming language, described by its proponents as "the language designed for building systems that really matter."

One of the most interesting applications of proof of correctness, based on the B method, is the development of software for the Paris Météor train. This is part of the Paris metro train system designed to carry up to 40,000 passengers per hour and per direction with an interval between trains as low as 85 seconds on peak hours. The safety-critical part of the software includes the running and stopping of every train, opening and closing of doors, electrical traction power, routes, speed of trains, and alarms from passengers. By the end of the project, 27,800 lemmas had been proven, with 92% proven automatically (with no human intervention). But here is the amazing part: the number of bugs in the Ada code found by testing on the host computer, the target computer, on site, and after the system was put into operation was—0. Zero, nada, none. Very impressive indeed.

Other formal method systems have been used for critical software projects, such as

- Development of a left ventricular assist device that helps the heart pump blood in those with congestive heart failure. The eventual goal is an artificial heart.
- "Conflict detection and resolution algorithms" for safety in air traffic control
- Development of the Tokeneer ID Station software to perform biometric verification of a human seeking access to a secure computing environment. Tokeneer is a hypothetical system promoted by NSA (National Security Agency) as a challenge problem for security researchers.

Formal Verification of Large Software Systems, Yin, X., and Knight, J., Proceedings of the NASA Formal Methods Symposium, April 13–15, 2010, Washington D.C., USA.

http://libre.adacore.com/academia/projects-single/echo

http://shemesh.larc.nasa.gov/fm/fm-atm-cdr.html

"Météor: A Successful Application of B in a Large Project," Behm, P., Benoit, P., Faivre, A., and Meynadier, J., *World Congress On Formal Methods in the Development of Computing Systems,* Toulouse, France, 1999, vol. 1709, pp. 369–387.

SECTION 2.3 REVIEW

TECHNIQUES

- Verify the correctness of a program segment that includes a loop statement.
- Compute gcd(a, b) using Euclid's algorithm.

MAIN IDEAS

- A loop invariant, proved by induction on the number of loop iterations, can be used to prove correctness of a program loop.
- The classic Euclidean algorithm for finding the greatest common divisor of two positive integers is provably correct.

EXERCISES 2.3

1. Let Q: $x^2 > x + 1$ where x is a positive integer. Assume that Q is true after the kth iteration of the following while loop; prove that Q is true after the next iteration.

```
while (x > 5) and (x < 40) do
   x = x + 1
end while
```

2. Let Q: $x! > 3^x$ where x is a positive integer. Assume that Q is true after the kth iteration of the following while loop; prove that Q is true after the next iteration.

```
while (x > 10) and (x < 30) do
   x = x + 2
end while
```

In Exercises 3–6, prove that the pseudocode program segment is correct by proving the given loop invariant Q and evaluating Q at loop termination.

3. Function to return the value of $x!$ for $x \geq 1$.

```
Factorial (positive integer x)
Local variables:
integers i, j
   i = 2
   j = 1
   while i ≠ x + 1 do
      j = j * i
      i = i + 1
   end while
   //j now has the value x!
   return j
end function Factorial
```

Q: $j = (i - 1)!$

4. Function to return the value of x^2 for $x \geq 1$.

```
Square (positive integer x)
Local variables:
integers i, j
   i = 1
   j = 1
   while i ≠ x do
      j = j + 2i + 1
      i = i + 1
   end while
```

```
    //j now has the value x^2
    return j
end function Square
```

$$Q\text{: } j = i^2$$

5. Function to return the value of x^y for $x, y \geq 1$.

```
Power (positive integer x, positive integer y)
Local variables:
integers i, j
    i = 1
    j = x
    while i ≠ y do
        j = j * x
        i = i + 1
    end while
    //j now has the value x^y
    return j
end function Power
```

$$Q\text{: } j = x^i$$

6. Function to compute and write out quotient q and remainder r when x is divided by y, $x \geq 0$, $y \geq 1$.

```
Divide (nonnegative integer x; positive integer y);
Local variables:
nonnegative integers q, r
    q = 0
    r = x
    while r ≥ y do
        q = q + 1
        r = r − y
    end while
    //q and r are now the quotient and remainder
    write("The quotient is" q "and the remainder is" r)
end function Divide
```

$$Q\text{: } x = q * y + r$$

For Exercises 7–12 use the Euclidean algorithm to find the greatest common divisor of the given numbers.

7. (308, 165)
8. (2420, 70)
9. (735, 90)
10. (8370, 465)
11. (1326, 252)
12. (1018215, 2695)
13. Following is the problem posed at the beginning of this chapter.

    The nonprofit organization at which you volunteer has received donations of 792 bars of soap and 400 bottles of shampoo. You want to create packages to distribute to homeless shelters such that each package contains the same number of shampoo bottles and each package contains the same number of bars of soap. How many packages can you create?

    Explain why the solution to this problem is the gcd(792, 400).

14. Concerning the question posed in Exercise 13:
    a. Use the Euclidean algorithm to find the number of packages.
    b. How many bottles of shampoo are in each package?
    c. How many bars of soap are in each package?

In Exercises 15–21, prove that the program segment is correct by finding and proving the appropriate loop invariant Q and evaluating Q at loop termination.

15. Function to return the value $x * y^n$ for $n \geq 0$.

```
Computation (integer x; integer y; nonnegative integer n)
Local variables:
integers i, j
   i = 0
   j = x
   while i ≠ n do
      j = j * y
      i = i + 1
   end while
   //j now has the value x * y^n
   return j
end function Computation
```

16. Function to return the value $x - y$ for $x, y \geq 0$.

```
Difference (nonnegative integer x; nonnegative integer y)
Local variables:
integers i, j
   i = 0
   j = x
   while i ≠ y do
      j = j - 1
      i = i + 1
   end while
   //j now has the value x - y
   return j
end function Difference
```

17. Function to return the value $(x + 1)^2$ for $x \geq 1$.

```
IncrementSquare (positive integer x)
Local variables:
integers i, j
   i = 1
   j = 4
   while i ≠ x do
      j = j + 2i + 3
      i = i + 1
   end while
   //j now has the value (x + 1)^2
   return j
end function IncrementSquare
```

18. Function to return the value 2^n for $n \geq 1$.

```
TwosPower (positive integer n)
Local variables:
integers i, j
   i = 1
   j = 2
   while i ≠ n do
      j = j * 2
      i = i + 1
   end while
   //j now has the value 2^n
   return j
end function TwosPower
```

19. Function to return the value $x * n!$ for $n \geq 1$.

```
AnotherOne (integer x; positive integer n)
Local variables:
integers i, j
   i = 1
   j = x
   while i ≠ n do
      j = j * (i + 1)
      i = i + 1
   end while
   //j now has the value x * n!
   return j
end function AnotherOne
```

20. Function to return the value of the polynomial $a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ at a given value of x.

```
Polly (real a_n; … ; real a_0; real x)
Local variables:
integers i, j
   i = n
   j = a_n
   while i ≠ 0 do
      j = j * x + a_{i-1}
      i = i - 1
   end while
   //j now has value of the polynomial evaluation
   return j
end function Polly
```

21. Function to return the maximum value from the first n entries $a[1], a[2], \ldots, a[n]$, $n \geq 1$, in an array of distinct integers.

```
ArrayMax (integers n, a[1], a[2], … , a[n])
Local variables:
integers i, j
   i = 1
```

```
    j = a[1]
    while i ≠ n do
        i = i + 1
        if a[i] > j then j = a[i]
    end while
    //j now has the value of the largest array element
    return j
end function ArrayMax
```

22. Following are four functions intended to return the value $a[1] + a[2] + \cdots + a[n]$ for $n \geq 1$ (the sum of the first n entries in an array of integers). For those that do not produce correct results, explain what goes wrong. For those that do produce correct results, do a proof of correctness.

a.
```
ArraySumA (integers n, a[1], a[2], … , a[n])
Local variables:
integers i, j
    i = 0
    j = 0
    while i ≤ n do
        i = i + 1
        j = j + a[i]
    end while
    //j now has the value a[1] + a[2] + ··· + a[n]
    return j
end function ArraySumA
```

b.
```
ArraySumB (integers n, a[1], a[2], … , a[n])
Local variables:
integers i, j
    i = 1
    j = 0
    while i ≤ n do
        j = j + a[i]
        i = i + 1
    end while
    //j now has the value a[1] + a[2] + ··· + a[n]
    return j
end function ArraySumB
```

c.
```
ArraySumC (integers n, a[1], a[2], … , a[n])
Local variables:
integers i, j
    i = 0
    j = 0
    while i ≤ n do
        j = j + a[i]
        i = i + 1
    end while
    //j now has the value a[1] + a[2] + ··· + a[n]
    return j
end function ArraySumC
```

d. *ArraySumD* (integers n, $a[1]$, $a[2]$, … , $a[n]$)
Local variables:
integers i, j
$i = 1$
$j = a[1]$
while $i \leq n$ **do**
$j = j + a[i + 1]$
$i = i + 1$
end while
//j now has the value $a[1] + a[2] + \cdots + a[n]$
return j
end function *ArraySumD*

Exercises 23–28 concern a variation of the Euclidean algorithm more suited to computer implementation. The original GCD algorithm relies on repeated integer divisions to compute remainders. The following variation, called the *binary GCD algorithm*, also uses divisions, but only by 2, plus subtraction and testing for parity (oddness or evenness). Given that the numbers are stored in the computer in binary form, these operations are simple and often done using built-in circuits. Testing for even/odd can be done using bitwise conjunction of N & 1, which results in 1 if and only if N is odd. Given an even N, division by 2 is easily accomplished by a 1-bit right shift operation, where all bits are shifted one place to the right (the rightmost bit disappears), and the leftmost bit is set to 0. (Multiplication by 2 is a 1-bit left shift.) Subtraction involves the 2's complement. As a result, the binary gcd algorithm, which avoids regular division, runs faster than the Euclidean algorithm, even though it does more (but simpler) steps. The binary GCD algorithm relies on three facts:

1. If both a and b are even, then $\gcd(a, b) = 2\gcd(a/2, b/2)$.
2. If a is even and b is odd, then $\gcd(a, b) = \gcd(a/2, b)$.
3. If a and b are both odd, and $a \geq b$, then $\gcd(a, b) = \gcd((a - b)/2, b)$.

23. To prove that if both a and b are even, then $\gcd(a, b) = 2\gcd(a/2, b/2)$, let a and b be even integers. Then 2 is a common factor of both a and b, so 2 is a factor of $\gcd(a, b)$. Let $2c = \gcd(a, b)$. Then

$$a = n(2c) \text{ and } b = m(2c)$$

$$a/2 = nc \text{ and } b/2 = mc$$

so $c \mid a/2$ and $c \mid b/2$. Finish this proof by showing that $c = \gcd(a/2, b/2)$.

24. To prove that if a is even and b is odd, then $\gcd(a, b) = \gcd(a/2, b)$, note that because b is odd, 2 is not a factor of b, hence not a factor of $\gcd(a, b)$. Therefore all contribution to $\gcd(a, b)$ comes from b and $a/2$, and $\gcd(a, b) = \gcd(a/2, b)$. Write an equation for $\gcd(a, b)$ when a is odd and b is even.

25. We want to prove that if a and b are both odd, and $a \geq b$, then $\gcd(a, b) = \gcd((a - b)/2, b)$. If a and b are both odd and $a \geq b$, then $\gcd(a, b) = \gcd(a - b, b)$ because, from the regular Euclidean algorithm, $\gcd(a, b)$ begins with $a = qb + r$, $0 \leq r < b$ and $\gcd(a - b, b)$ begins with $a - b = (q - 1)b + r$, $0 \leq r < b$. The next step in either case is to divide b by r, so the two final answers will be the same. Finish this proof by showing that $\gcd(a - b, b) = \gcd((a - b)/2, b)$.